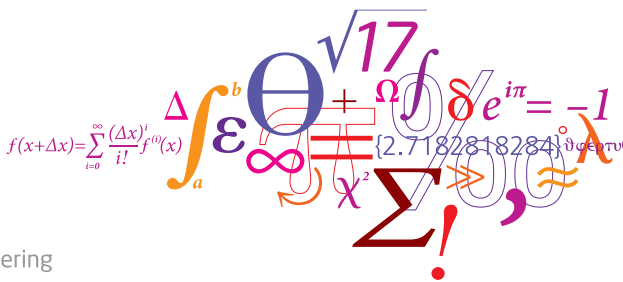
Lecture 4 - Classification



DTU Management Engineering
Department of Management Engineering

Learning Objectives

- To explain the basic setting of a classification problem and its difference from regression
- To formulate a logistic regression model and explain the basic concept behind parameter estimation using MLE
- To train a logistic regression model using SKLearn, make predictions, interpret coefficients and analyze model performance
- To explain the basic concepts behind the maximal margin classifier, support vector classifier, and support vector machine; to explain the role of the kernel function and regularization parameter
- To train a support vector machine for classification using SKlearn and make predictions

Outline

- Introduction
- Models
  - Logistic Regression
  - Support Vector Machines

Introduction

- Linear regression $\rightarrow$ Predicting a **continuous** dependent variable from a set of independent variables.
- Classification $\rightarrow$ Predicting a **discrete** dependent variable from a set of independent variables.
- Identifying to which of a set of classes, $\{\pi_1, ..., \pi_g\}$, a new observation belongs, on the basis of a training set of data containing observations whose classes are known.

Classification

Examples:

- Vehicle identification - determine type of vehicle from camera
- Incident detection - determine incident/no-incident from flow and speed data
- Spam filter - Allocate emails to spam/no-spam
- Handwritten Digit Recognition - Allocate the images of handwritten zip codes on mail to digits

Example: Vehicle Type Identification

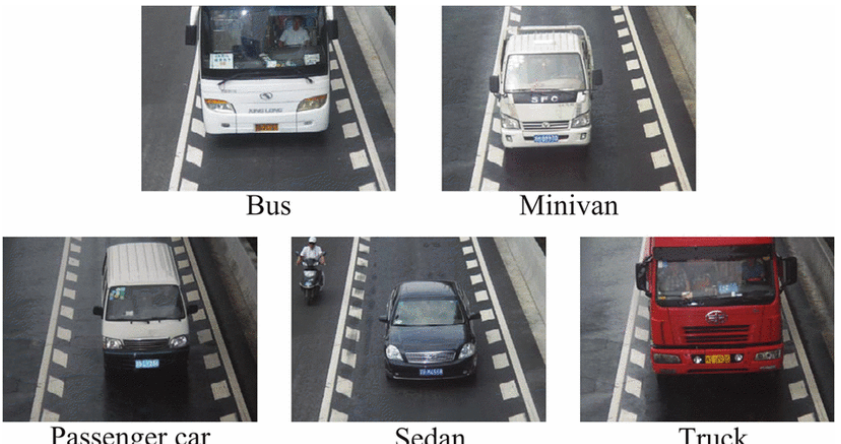


Image source: Dong, Zhen et al. "Vehicle Type Classification Using Unsupervised Convolutional Neural Network." IEEE, 2014. 172–177. Print.

Example: Vehicle Type Identification

- Images were captured from traffic cameras
- The task is to identify the types of vehicles in these images
- Each image represents one observation
- Dependent variable $Y \in \{Bus, MiniVan, PassengerCar, Sedan, Truck\}$
- Applications: traffic volume estimation, illegal vehical type identification

Regression for Discrete Dependent Variable?

- Can we directly use linear regression models to predict the discrete dependent variable?
- Linear regression model:

$$\mathbf{Y} = \mathbf{X}\beta + \boldsymbol{\epsilon}$$

- We used least squares method to estimate the parameters β

Regression for Discrete Dependent Variable?

- Can we directly use linear regression models to predict the discrete dependent variable?
- Linear regression model:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

- We used least squares method to estimate the parameters $\boldsymbol{\beta}$
- This model predicts continuous variable with values ranging from $-\infty$ to ∞ i.e, $Y \in (-\infty, \infty)$
- We need another way to use a 'regression like' approach for discrete dependent variables.

Binary Logistic Regression

- Dependent variable, Y, is restricted to two values.
- Examples: $\{\text{Incident}, \text{NoIncident}\}$, $\{\text{Spam}, \text{NotSpam}\}$, $\{\text{Employed}, \text{NotEmployed}\}$
- Denote the two cases as 0 and 1

Binary Logistic Regression

- Dependent variable, Y, is restricted to two values.
- Examples: $\{\text{Incident}, \text{NoIncident}\}$, $\{\text{Spam}, \text{NotSpam}\}$, $\{\text{Employed}, \text{NotEmployed}\}$
- Denote the two cases as 0 and 1
- For a given observation, we want to estimate the **probability** that Y belongs to either case.

Binary Dependent Variables

- We need only focus on predicting $p(y_i = 1)$

  –> $p(y_i = 0) = 1 - p(y_i = 1)$

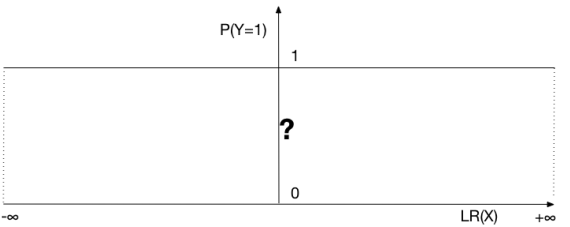
Binary Dependent Variables

- We need only focus on predicting $p(y_i = 1)$

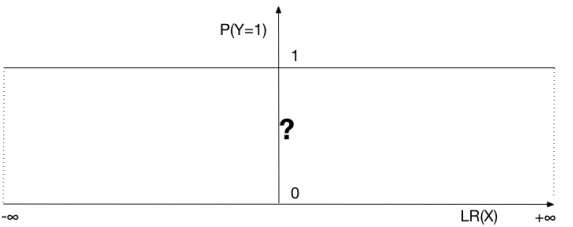
  $->$ $p(y_i = 0) = 1 - p(y_i = 1)$

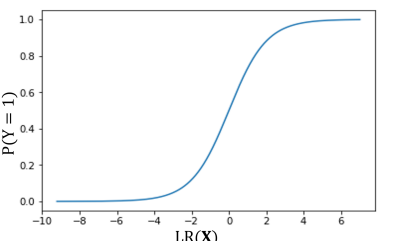
- Linear regression model still cannot be used since the probability $p(y_i = 1), \in [0, 1]$

- Need to establish a function that links $p(y_i = 1)$ to a continuous variable that ranges from $-\infty$ to $+\infty$, possibly obtained by a linear model (LR(X)).



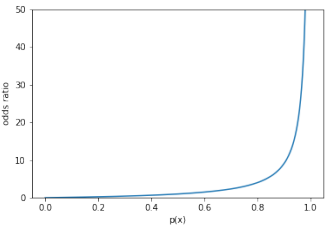
- Need to establish a function that links $p(y_i = 1)$ to a continuous variable that ranges from $-\infty$ to $+\infty$, possibly obtained by a linear model (LR(X)).





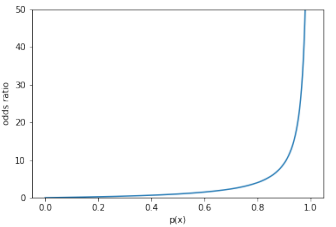
Odds ratio

$$odds = \frac{p(y_i = 1)}{1 - p(y_i = 1)}$$



Odds ratio

$$odds = \frac{p(y_i = 1)}{1 - p(y_i = 1)}$$



- not symmetric
- $[0, +\infty[$, but it should be $]-\infty, +\infty[$

Logit model

Take the natural logarithm of the odds ratio

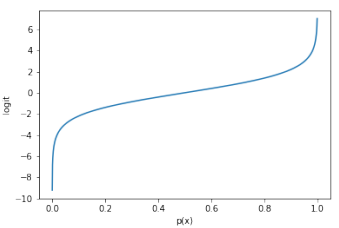
$$\text{logit}(p(y_i = 1)) = \ln(odds) = \ln\left(\frac{p(y_i = 1)}{1 - p(y_i = 1)}\right)$$

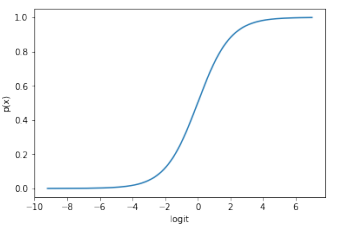
Logit model

Take the natural logarithm of the odds ratio

$$\text{logit}(p(y_i = 1)) = \ln(odds) = \ln\left(\frac{p(y_i = 1)}{1 - p(y_i = 1)}\right)$$

Log odds ratio (*logit*) provides a monotonically increasing function of $p(y_i = 1)$ with range $(-\infty, \infty)$





Binary Logit Model

Now we can use a linear function of the predictors to model the log odds ratio (*logit*)

$$\text{logit}(p(y_i = 1)) = \ln\left(\frac{p(y_i = 1)}{1 - p(y_i = 1)}\right) = \boldsymbol{\beta}^T \mathbf{x}_i$$

Binary Logit Model

Now we can use a linear function of the predictors to model the log odds ratio (*logit*)

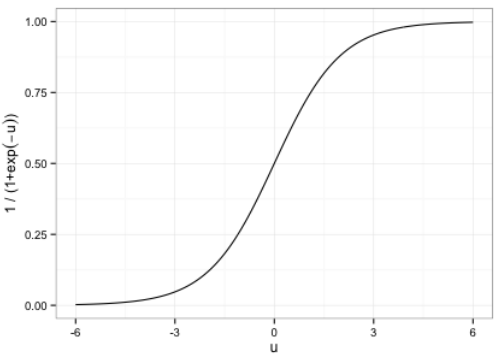
$$\text{logit}(p(y_i = 1)) = \ln\left(\frac{p(y_i = 1)}{1 - p(y_i = 1)}\right) = \boldsymbol{\beta}^T \mathbf{x}_i$$

From the predicted value of the *logit*, we can obtain the probability of $y_i = 1$ through the *logistic sigmoid function*:

$$p(y_i = 1) = \frac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x_i})}$$

Logistic Sigmoid Function

$$p(y_i = 1) = \frac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x_i})}$$



Parameter Interpretation

- β_0 is the intercept
- $\beta_1, \beta_2, ..., \beta_r$ are the coefficients
- The sign of β_j indicates the direction of change in $p(y)$ with respect to change in the variable X_j

Parameter Interpretation

- β_0 is the intercept
- $\beta_1, \beta_2, ..., \beta_r$ are the coefficients
- The sign of β_j indicates the direction of change in $p(y)$ with respect to change in the variable X_j
- The magnitude of β_j affects the steepness of the sigmoid function

Model Estimation

- We need to estimate β

Model Estimation

- We need to estimate $\boldsymbol{\beta}$
- We want to choose parameters $\boldsymbol{\beta} = [\beta_0, \beta_1, ..., \beta_r]^T$ such that the joint probability of $P(\boldsymbol{Y} = \mathbf{y})$ for all observations is maximized

Model Estimation

- We need to estimate $\boldsymbol{\beta}$
- We want to choose parameters $\boldsymbol{\beta} = [\beta_0, \beta_1, ..., \beta_r]^T$ such that the joint probability of $P(\boldsymbol{Y} = \mathbf{y})$ for all observations is maximized
- Example
  - Obs 1: $\mathbf{x_1}, y_1 = 1$; $p(y_1 = 1) = p_1 = \dfrac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x_1})}$
  - Obs 2: $\mathbf{x_2}, y_2 = 0$; $p(y_2 = 0) = p_2 = 1 - \left(\dfrac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x_2})} \right)$
  - $L(\boldsymbol{\beta}) = p_1 \times p_2$

Model Estimation

- We need to estimate $\boldsymbol{\beta}$
- We want to choose parameters $\boldsymbol{\beta} = [\beta_0, \beta_1, ..., \beta_r]^T$ such that the joint probability of $P(\boldsymbol{Y} = \mathbf{y})$ for all observations is maximized
- Example
  - Obs 1: $\mathbf{x_1}, y_1 = 1$; $p(y_1 = 1) = p_1 = \dfrac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x_1})}$
  - Obs 2: $\mathbf{x_2}, y_2 = 0$; $p(y_2 = 0) = p_2 = 1 - \left(\dfrac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x_2})} \right)$
  - $L(\boldsymbol{\beta}) = p_1 \times p_2$
- The function of parameters $L(\boldsymbol{\beta})$ that we want to maximize is called the *Likelihood Function*

Maximum Likelihood

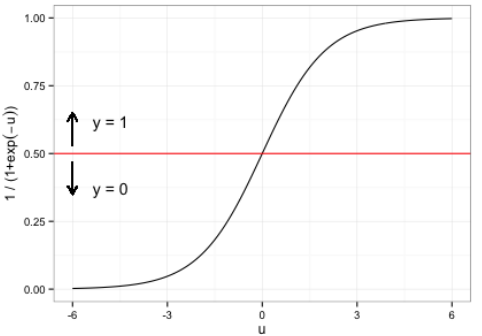
- Typically to maximize the log likelihood we differentiate the function with respect to the parameters and set it equal to zero
- In case of the logistic regression there is no closed form solution due to the non-linearity introduced by exponentials inside the sum
- We need to use numerical methods for finding the parameter values that maximize the log likelihood

Prediction

For a test observation or point i, once we obtain the probability of $y_i = 1$, we assign a prediction value such that

$$\begin{cases} y_i = 1 & \text{if } \ p(y_i) > 0.5 \\ y_i = 0 & \text{otherwise} \end{cases}$$

Logistic Regression: Decision Boundary



Evaluating Classifiers

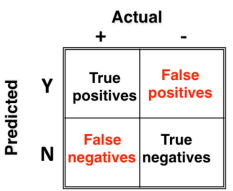
- Fill a *confusion matrix* (also known as *success matrix*) contrasting the actual classification with the predicted on the training set

		Actual Class	
		Dog	Cat
Predicted Class	Dog	88	12
	Cat	10	90

Evaluating Classifiers

- Fill a *confusion matrix* (also known as *success matrix*) contrasting the actual classification with the predicted on the training set

		Actual Class	
		Dog	Cat
Predicted Class	Dog	88	12
	Cat	10	90



From https://www.svds.com/the-basics-of-classifier-evaluation-part-1/

Evaluating Classifiers

- Accuracy

$$\frac{\#\text{Correct predictions}}{\#\text{Total predictions}}$$

Evaluating Classifiers

- Accuracy

$$\frac{\#\text{Correct predictions}}{\#\text{Total predictions}}$$

- Precision=Positive predictive value $= \frac{TP}{TP+FP}$
- Recall = Sensitivity = true positive rate $= \frac{TP}{TP+FN}$

Evaluating Classifiers

- Accuracy

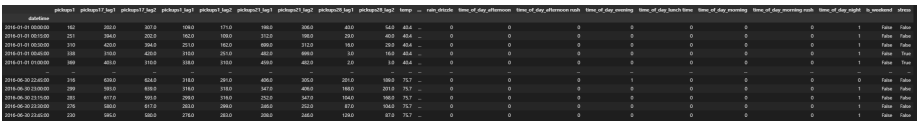
$$\frac{\#\text{Correct predictions}}{\#\text{Total predictions}}$$

- Precision=Positive predictive value $= \frac{TP}{TP+FP}$
- Recall = Sensitivity = true positive rate $= \frac{TP}{TP+FN}$
- F1 score is the harmonic mean of precision and recall:

$$F1 = 2 * \frac{precision * recall}{precision + recall}$$

Data and problem for today

- New-york city taxi data from last week (with additional weather data and dummies)
- We now label each interval as either being a 'stress' situation or not (high pickups) and we wish to make a binary prediction (stress or not stress)



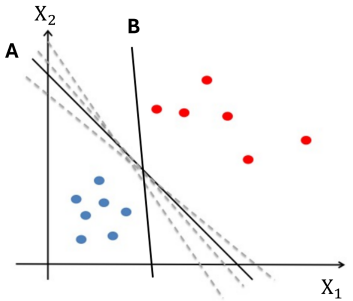
datetime	pickups1	pickups17_lag1	pickups17_lag2	pickups1_lag1	pickups1_lag2	pickups21_lag1	pickups21_lag2	pickups28_lag1	pickups28_lag2	temp	...	rain_drizzle	time_of_day_afternoon	time_of_day_afternoon rush	time_of_day_evening	time_of_day_lunch time	time_of_day_morning	time_of_day_morning rush	time_of_day_night	is_weekend	stress
2016-01-01 00:00:00	162	202.0	207.0	109.0	171.0	198.0	206.0	43.0	54.0	40.4	...	0	0	0	0	0	0	0	1	False	False
2016-01-01 00:15:00	251	394.0	202.0	162.0	109.0	312.0	198.0	23.0	43.0	40.4	...	0	0	0	0	0	0	0	1	False	False
2016-01-01 00:30:00	310	420.0	394.0	251.0	162.0	699.0	312.0	19.0	23.0	40.4	...	0	0	0	0	0	0	0	1	False	False
2016-01-01 00:45:00	336	310.0	420.0	310.0	251.0	462.0	699.0	3.0	19.0	40.4	...	0	0	0	0	0	0	0	1	False	True
2016-01-01 01:00:00	369	403.0	310.0	336.0	310.0	459.0	462.0	2.0	3.0	40.4	...	0	0	0	0	0	0	0	1	False	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2016-06-30 22:45:00	316	639.0	624.0	318.0	291.0	406.0	305.0	201.0	199.0	75.7	...	0	0	0	1	0	0	0	0	False	False
2016-06-30 23:00:00	299	593.0	639.0	316.0	318.0	347.0	406.0	166.0	201.0	75.7	...	0	0	0	0	0	0	0	1	False	False
2016-06-30 23:15:00	283	617.0	593.0	299.0	316.0	252.0	347.0	154.0	166.0	75.7	...	0	0	0	0	0	0	0	1	False	False
2016-06-30 23:30:00	276	580.0	617.0	283.0	299.0	246.0	252.0	87.0	154.0	75.7	...	0	0	0	0	0	0	0	1	False	False
2016-06-30 23:45:00	230	595.0	580.0	276.0	283.0	209.0	246.0	129.0	87.0	75.7	...	0	0	0	0	0	0	0	1	False	False

```
Index(['pickups1', 'pickups17_lag1', 'pickups17_lag2', 'pickups1_lag1',
       'pickups1_lag2', 'pickups21_lag1', 'pickups21_lag2', 'pickups28_lag1',
       'pickups28_lag2', 'temp', 'prcp', 'fog', 'rain_drizzle',
       'time_of_day_afternoon', 'time_of_day_afternoon rush',
       'time_of_day_evening', 'time_of_day_lunch time', 'time_of_day_morning',
       'time_of_day_morning rush', 'time_of_day_night', 'is_weekend',
       'stress'],
      dtype='object')
```

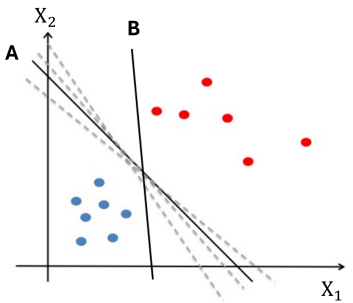
Playtime!

- Open the "4-Classification.ipynb" notebook
- Do Part 1
- Estimated duration: 45 min

Support Vector Machines - Introduction

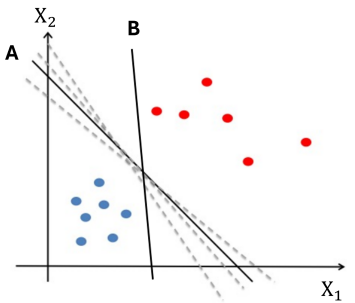


Support Vector Machines - Introduction



$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 \begin{cases} = 0 & \text{if } \mathbf{x} \text{ is on the line} \\ < 0 & \text{if } \mathbf{x} \text{ is below} \\ > 0 & \text{if } \mathbf{x} \text{ is above} \end{cases}$$

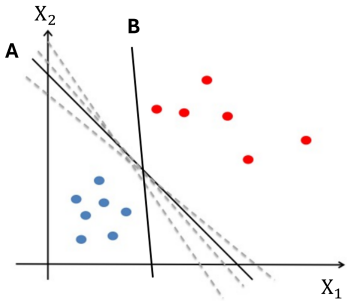
Support Vector Machines - Introduction



$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 \begin{cases} = 0 & \text{if x is on the line} \\ < 0 & \text{if x is below} \\ > 0 & \text{if x is above} \end{cases}$$

- Find a line that separates the observations belonging to different classes.
- Would you choose line A or line B? Why?

Support Vector Machines - Introduction

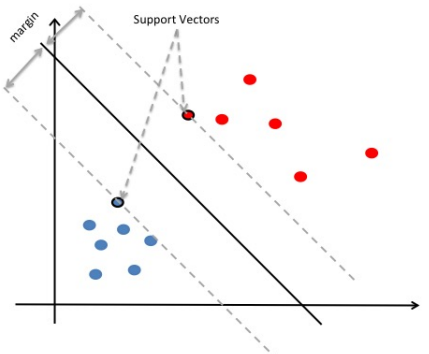


$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 \begin{cases} = 0 & \text{if } \mathbf{x} \text{ is on the line} \\ < 0 & \text{if } \mathbf{x} \text{ is below} \\ > 0 & \text{if } \mathbf{x} \text{ is above} \end{cases}$$

- Find a line that separates the observations belonging to different classes.
- Would you choose line A or line B? Why?
- **Note**: for mathematical convenience, y will now be {-1, 1} (instead of {0, 1} from before)

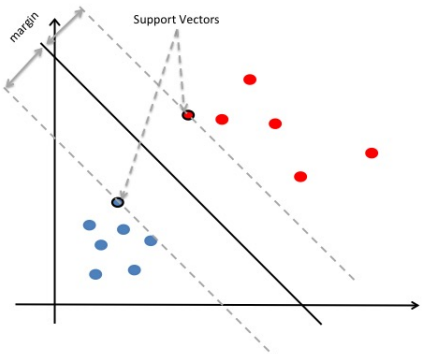
Support Vector Machines - Support Vectors

Goal in SVM: maximize the separation margin.



Support Vector Machines - Support Vectors

Goal in SVM: maximize the separation margin.



- Being further away from the margin implies we have more confidence in the classification

Support Vector Machines - Formalization

A hyperplane is defined by parameters $\beta_0, \boldsymbol{\beta}$ such that

$$\beta_0 + \boldsymbol{\beta}^T \mathbf{x} \begin{cases} = 0 & \text{if } \mathbf{x} \text{ lies on the plane} \\ < 0 & \text{if } \mathbf{x} \text{ on one side} \\ > 0 & \text{if } \mathbf{x} \text{ on the other side} \end{cases}$$

Support Vector Machines - Formalization

A hyperplane is defined by parameters $\beta_0, \boldsymbol{\beta}$ such that

$$\beta_0 + \boldsymbol{\beta}^T \mathbf{x} \begin{cases} = 0 & \text{if } \mathbf{x} \text{ lies on the plane} \\ < 0 & \text{if } \mathbf{x} \text{ on one side} \\ > 0 & \text{if } \mathbf{x} \text{ on the other side} \end{cases}$$

Rule: $\text{sign}(\beta_0 + \boldsymbol{\beta}^T \mathbf{x})$ tells us the class.

Support Vector Machines - Formalization

A hyperplane is defined by parameters $\beta_0, \boldsymbol{\beta}$ such that

$$\beta_0 + \boldsymbol{\beta}^T \mathbf{x} \begin{cases} = 0 & \text{if } \mathbf{x} \text{ lies on the plane} \\ < 0 & \text{if } \mathbf{x} \text{ on one side} \\ > 0 & \text{if } \mathbf{x} \text{ on the other side} \end{cases}$$

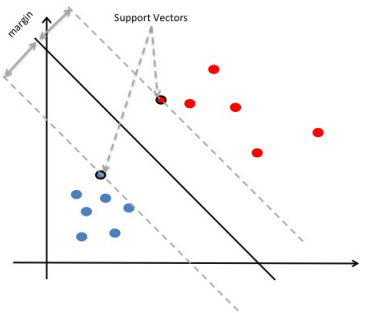
Rule: $\text{sign}(\beta_0 + \boldsymbol{\beta}^T \mathbf{x})$ tells us the class.

Maximal Margin Classifier: $\beta_0, \boldsymbol{\beta}$ correspond to the *margin-maximizer* hyperplane

$$\max_{\beta_0, \boldsymbol{\beta}} M \text{ s.t.}$$
$$y_j(\beta_0 + \boldsymbol{\beta}^T \mathbf{x}_j) \geq M, \forall j$$
$$||\boldsymbol{\beta}||^2 = 1$$

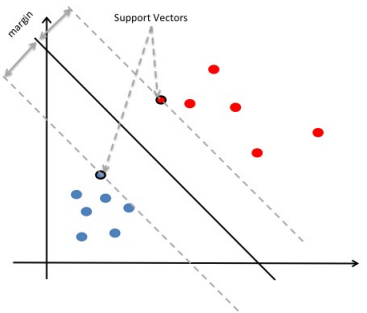
Support Vector Machines - Formalization

- After calculation, we find that $\boldsymbol{\beta} = \sum_{j \in \mathcal{S}} \alpha_j y_j \mathbf{x}_j$.



Support Vector Machines - Formalization

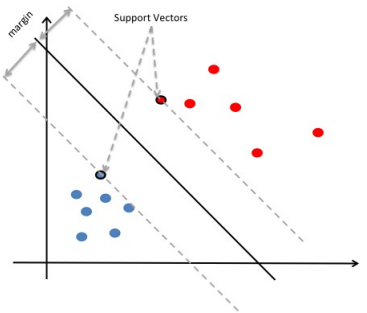
- After calculation, we find that $\boldsymbol{\beta} = \sum_{j \in \mathcal{S}} \alpha_j y_j \mathbf{x}_j$.



- The plane only depends on the support vectors $\mathcal{S}$ (subset of the training set) - robustness

Support Vector Machines - Formalization

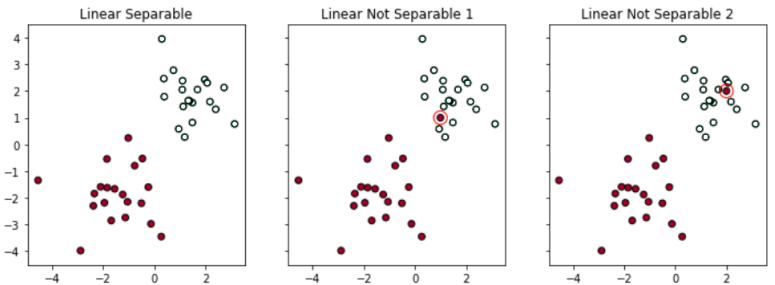
- After calculation, we find that $\boldsymbol{\beta} = \sum_{j \in \mathcal{S}} \alpha_j y_j \mathbf{x}_j$.



- The plane only depends on the support vectors $\mathcal{S}$ (subset of the training set) - robustness
- β_0 can be obtained by solving $\alpha_j \left[y_j(\beta_0 + \boldsymbol{\beta}^T \mathbf{x}_j) - 1 \right] = 0$ for any $\mathbf{x}_j, j \in \mathcal{S}$

Support Vector Machines - Hard vs Soft margin

What if the data is not linearly separable?



- *Support Vector Classifier*: reformulate optimization problem to "tolerate" a few points getting misclassified or being within the margin
- Tolerance parameter C controls the number of points that misclassified or on the wrong side of the margin

Support Vector Machines - Hard vs Soft margin

Tolerance parameter C tries to balance the trade-off between finding a line that maximizes the margin and minimizes the misclassification

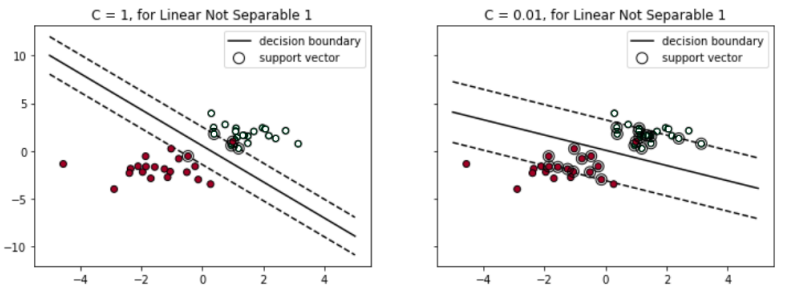
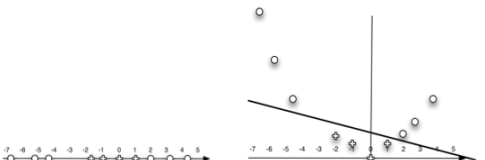


Image credit: `https://medium.com/bite-sized-machine-learning/support-vector-machine-explained-soft-margin-kernel-tricks-3728dfb92cee`

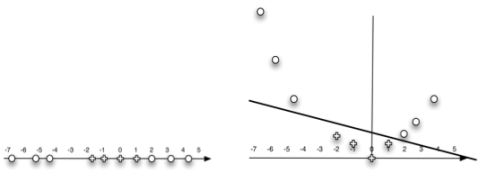
Support Vector Machines - Kernel trick

What if the data is not linearly separable?



Support Vector Machines - Kernel trick

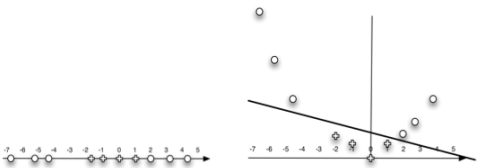
What if the data is not linearly separable?



- We can find a mapping $\boldsymbol{\phi}(X)$ into a higher dimension where all objects are linearly separable, in this case $\boldsymbol{\phi}(X) = (X, X^2)$

Support Vector Machines - Kernel trick

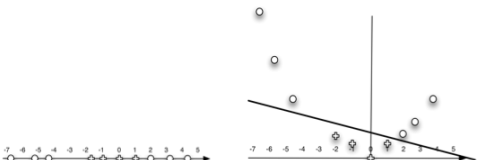
What if the data is not linearly separable?



- We can find a mapping $\boldsymbol{\phi}(X)$ into a higher dimension where all objects are linearly separable, in this case $\boldsymbol{\phi}(X) = (X, X^2)$
- Another example: $\mathbf{X} = (X_1, X_2)$; $\boldsymbol{\phi}(\mathbf{X}) = (X_1, X_2, X_1X_2, X_2^2, X_1^2)$

Support Vector Machines - Kernel trick

What if the data is not linearly separable?



- We can find a mapping $\boldsymbol{\phi}(X)$ into a higher dimension where all objects are linearly separable, in this case $\boldsymbol{\phi}(X) = (X, X^2)$
- Another example: $\mathbf{X} = (X_1, X_2)$; $\boldsymbol{\phi}(\mathbf{X}) = (X_1, X_2, X_1X_2, X_2^2, X_1^2)$
- How do we define this feature transformation? Kernel functions !

Support Vector Machines - Kernel functions

- Recall the equation of the separating hyperplane in the support vector classifier:

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

where $\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \sum_{k=1}^{p} x_{ik} x_{jk}$

Support Vector Machines - Kernel functions

- Recall the equation of the separating hyperplane in the support vector classifier:

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

  where $\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \sum_{k=1}^{p} x_{ik} x_{jk}$

- When we enlarge the feature space, we have

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$$

Support Vector Machines - Kernel functions

- Recall the equation of the separating hyperplane in the support vector classifier:

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

  where $\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \sum_{k=1}^{p} x_{ik} x_{jk}$

- When we enlarge the feature space, we have

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$$

- The kernel function K computes these inner products in the transformed space

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$$

Support Vector Machines - Kernel functions

- Recall the equation of the separating hyperplane in the support vector classifier:

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

where $\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \sum_{k=1}^{p} x_{ik} x_{jk}$

- When we enlarge the feature space, we have

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$$

- The kernel function K computes these inner products in the transformed space

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$$

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

Support Vector Machines - Kernel functions

- Recall the equation of the separating hyperplane in the support vector classifier:

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

where $\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \sum_{k=1}^{p} x_{ik} x_{jk}$

- When we enlarge the feature space, we have

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$$

- The kernel function K computes these inner products in the transformed space

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$$

$$f(\mathbf{x}_i) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_j y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

- If we have K we do not need to know or compute $\boldsymbol{\phi}(\mathbf{x})$ at all!

Support Vector Machines - Kernel functions (contd.)

- A simple example: consider the case with two features X_1 and X_2, and two observations $\mathbf{x}_i = (x_{i1}, x_{i2})$ and $\mathbf{x}_j = (x_{j1}, x_{j2})$

Support Vector Machines - Kernel functions (contd.)

- A simple example: consider the case with two features X_1 and X_2, and two observations $\mathbf{x}_i = (x_{i1}, x_{i2})$ and $\mathbf{x}_j = (x_{j1}, x_{j2})$

$$K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \langle \mathbf{x}_i, \mathbf{x}_j \rangle)^2$$

$$= (1 + x_{i1}x_{j1} + x_{i2}x_{j2})^2$$

$$= \left(1 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} + (x_{i1}x_{j1})^2 + (x_{i2}x_{j2})^2 + 2x_{i1}x_{j1}x_{i2}x_{j2}\right)$$

Support Vector Machines - Kernel functions (contd.)

- A simple example: consider the case with two features X_1 and X_2, and two observations $\mathbf{x}_i = (x_{i1}, x_{i2})$ and $\mathbf{x}_j = (x_{j1}, x_{j2})$

$$K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \langle \mathbf{x}_i, \mathbf{x}_j \rangle)^2$$

$$= (1 + x_{i1}x_{j1} + x_{i2}x_{j2})^2$$

$$= \left(1 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} + (x_{i1}x_{j1})^2 + (x_{i2}x_{j2})^2 + 2x_{i1}x_{j1}x_{i2}x_{j2}\right)$$

- If we were to define our enlarged feature space as

$$\boldsymbol{\phi}(\mathbf{x}_i) = (1, \sqrt{2}x_{i1}, \sqrt{2}x_{i2}, x_{i1}^2, x_{i2}^2, \sqrt{2}x_{i1}x_{i2})$$

we see that $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle$!

Support Vector Machines - Kernel functions (contd.)

- *Linear* kernel: original support vector classifier

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

Support Vector Machines - Kernel functions (contd.)

- *Linear* kernel: original support vector classifier

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

- *Polynomial* kernel of degree d: Generates new features through polynomial combinations of existing features (degree d)

$$K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \langle \mathbf{x}_i, \mathbf{x}_j \rangle)^d$$

Support Vector Machines - Kernel functions (contd.)

- *Linear* kernel: original support vector classifier

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

- *Polynomial* kernel of degree d: Generates new features through polynomial combinations of existing features (degree d)

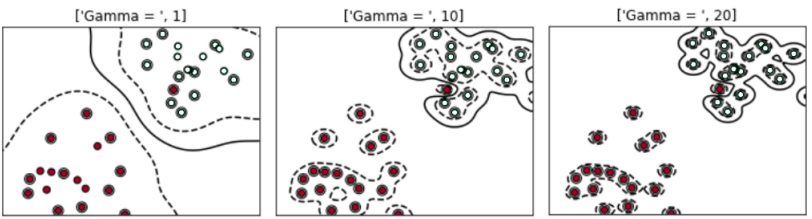
$$K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \langle \mathbf{x}_i, \mathbf{x}_j \rangle)^d$$

- *Gaussian Radial basis function* kernel: value depends on the distance between the two points point

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) = \exp(-\gamma \sum_{k=1}^{p} (x_{ik} - x_{jk})^2)$$

Support Vector Machines - RBF kernel

- Parameter γ (gamma) controls how much nearby points influence each other
- In practice, controls how "wiggling" the decision boundary is



Support Vector Machines - Conclusion

- SVMs learn the function of class separation as in logistic regression.
- The objective function for finding the best separation of classes is based on maximizing the separation margin.
- SVMs do not model the distribution of the data rather only find the support vectors.

Playtime!

- Open the "4-Classification.ipynb" notebook
- Do Part 2
- Estimated duration: 20 min